// BLUEPRINT IMPLEMENTATION GUIDE — PART 3

PROJECT ANOMALY — Animation, Audio & Polish

Covers the 23 systems added by the TDD animation pipeline update: Anim Blueprints, Blend Spaces, Anim Notifies, IK, footsteps, hit reactions, ragdoll, post process effects, Enhanced Input setup, Sound Classes, and the three missing UI screens.

CONTINUES PARTS 1 & 2 | 23 NEW SYSTEMS | ANIMATION PIPELINE | UE 5.4

// 01 ENHANCED INPUT — IMC SETUP (MISSING FROM PART 1)

> Part 1 used Enhanced Input event nodes but never showed how to create the Input Actions, the Mapping Context, or how to register them. Without this step nothing fires. Do this before any input works.

SETUP Create Input Actions and Mapping Context

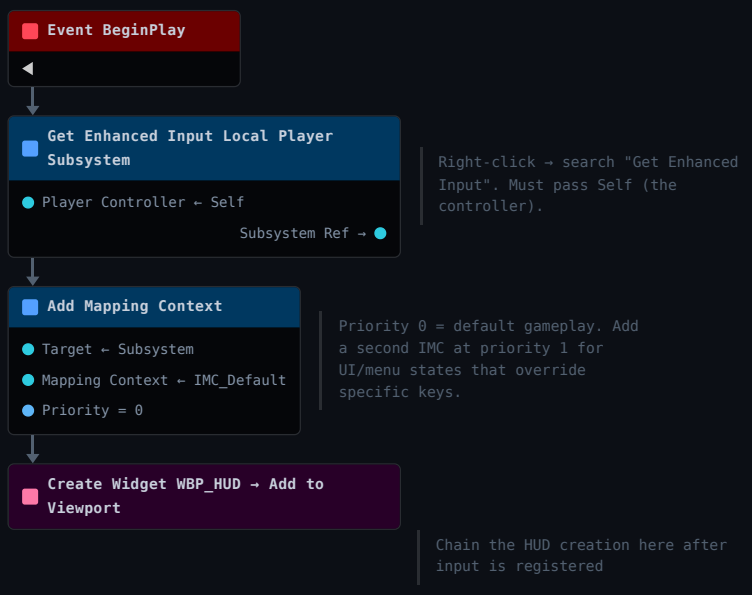
In `Input/` folder → right-click → **Input → Input Action**. Create one per action. For each IA_ asset, set **Value Type**:

ASSET	VALUE TYPE	TRIGGER
IA_Fire	Digital (bool)	Started + Completed (for AR StopFire)
IA_Parry	Digital (bool)	Started
IA_Grapple	Digital (bool)	Started
IA_Dash	Digital (bool)	Started
IA_Utility	Digital (bool)	Started
IA_Skill	Digital (bool)	Started
IA_WeaponWheel	Digital (bool)	Started + Completed
IA_Jump	Digital (bool)	Started
IA_Move	Axis2D (Vector2D)	Triggered (continuous)
IA_Look	Axis2D (Vector2D)	Triggered (continuous)

Then right-click → **Input → Input Mapping Context**. Name it `IMC_Default`. Open it → click **+** to add mappings → assign each IA_ to its key.

GRAPH BP_PlayerController BeginPlay — register the IMC

BEGINPLAY — WITHOUT THIS, ALL INPUT IS SILENT



FLOW SUMMARY — ENHANCED INPUT SETUP

1	**IA_ = the action, IMC_ = the key binding.**	They're separate assets so keys can be rebound (edit IMC_) without touching any Blueprint that uses the IA_.
2	**Add Mapping Context must run before any input fires.**	PlayerController BeginPlay is the correct place — it runs before the player character spawns.
3	**Priority resolves conflicts.**	If two IMCs both map the same key, the higher Priority number wins. Use this to override gameplay keys with menu keys when UI is open.

🔧 DEBUG & TEST — ENHANCED INPUT

HOW TO TEST

- **Verify IMC registered:** Add `Print String "INPUT READY"` immediately after Add Mapping Context in BeginPlay. If it doesn't print, the BeginPlay isn't running — check BP_PlayerController is assigned in BP_GM_Anomaly's Player Controller Class.
- **Test each IA_ fires:** In BP_PlayerCharacter, for each input event node, add a temporary `Print String` with the action name. Press every key — all 10 should print.
- **Test Axis2D for IA_Move:** Print the ActionValue from IA_Move. Moving forward should print approximately (0, 1). Moving left should print (-1, 0). Use Get Action Value → Break Vector 2D to inspect X and Y.

EXPECTED RESULTS

- ✓ PASS — "INPUT READY" prints at game start
- ✓ PASS — Every key press prints the matching action name
- ✓ PASS — IA_Move ActionValue changes with WASD input
- ✗ FAIL — No input fires at all: Add Mapping Context is missing. Also check Project Settings → Input → Default Input Component Class is set to EnhancedInputComponent.
- ✗ FAIL — One key fires multiple actions: two IA_ assets are bound to the same key in IMC_Default. Open the IMC_ and remove the duplicate.

ABP_PLAYER — ANIMATION BLUEPRINT

The Anim Blueprint controls what animation the player plays every frame. It reads movement data from the Character Movement Component and drives a Blend Space. Montages layer on top for actions like grappling or dashing.

CREATE — Create ABP_Player

In `Player/Animation/` → right-click → **Animation → Animation Blueprint**. Select the player's skeleton (`SK_Player`). Name it `ABP_Player`. Open it — you'll see two graphs: **AnimGraph** (controls poses) and **Event Graph** (updates variables). Assign it to BP_PlayerCharacter by selecting the Mesh component → Details → Anim Class → `ABP_Player`.

GRAPH — Event Graph — update variables every frame

EVENT BLUEPRINT UPDATE ANIMATION — RUNS EVERY FRAME

Event Blueprint Update Animation
- DeltaTime →
- ◀

The ABP's tick event — runs before poses are evaluated

↓

Try Get Pawn Owner → Cast to BP_PlayerCharacter
- ◀ Success →
- Typed ref →

Always check pawn is valid — ABP can tick before the pawn fully exists

↓

Get Character Movement → Velocity → VSize
- → SET Speed

Speed drives the X axis of BS_Player_Locomotion

↓

Calculate Direction (Velocity, Actor Rotation)
- → SET Direction

Returns -180 to 180. Right-click → search "Calculate Direction"

↓

Is Moving On Ground → SET bIsGrounded
- → SET bIsGrounded

↓

IsEnraged (BPI_Rageable) → SET bIsEnraged
- Target ← Pawn Owner

Use interface so ABP never has a hard ref to AC_RageComponent

ANIM GRAPH — AnimGraph — Blend Space + Slot for montages

Switch to the **AnimGraph** tab. This is where poses connect. Build this graph left-to-right ending at the **Output Pose** node:

ANIMGRAPH — LOCOMOTION INTO MONTAGE SLOT INTO OUTPUT

Blend Space Player: BS_Player_Locomotion
- Speed ← Speed variable
- Direction ← Direction variable
- Pose →

Continuously blends locomotion animations based on speed and strafe direction

↓

Slot "DefaultSlot"
- Source ← BS pose
- Pose →

Montages (Play Anim Montage calls) blend in through this slot. When no montage is playing, it passes through the source pose unchanged.

↓

Output Pose

The final pose applied to the mesh every frame

📌 **Creating BS_Player_Locomotion:** Right-click in `Player/Animation/BlendSpaces/` → Animation → Blend Space 1D (or 2D). Open it. Drag animation sequences onto the grid. Set the axis min/max and add sample points. For 2D: Speed on horizontal (0–600), Direction on vertical (-180 to 180).

FLOW SUMMARY — ABP_PLAYER

1. **Event Graph updates variables each tick.** It never sets poses directly — it only reads data and stores it in variables that the AnimGraph uses.
2. **AnimGraph reads variables to set poses.** BS_Player_Locomotion blends idle/walk/run/strafe animations based on Speed and Direction. This runs continuously every frame.
3. **DefaultSlot layers montages on top.** When Play Anim Montage is called (e.g. for parry or grapple), the montage blends into this slot and overrides the locomotion blend for its duration, then blends back out.
4. **ABP_ never talks to components.** It reads from the pawn via Try Get Pawn Owner and uses the interface for rage state. No direct component references — safe against component changes.

🔧 DEBUG & TEST — ABP_PLAYER

HOW TO TEST

- **Open Anim Preview:** With the level running, select the player in the Outliner → Details → Skeletal Mesh → click the magnifying glass. The Animation Editor opens showing the live ABP_ state.
- **Watch variable values:** In the ABP_ Event Graph, hover over the Speed variable during PIE. The tooltip shows the live value. Walk forward — Speed should increase to ~600.
- **Test blend space:** Walk sideways — Direction should change and a different locomotion animation should blend in. If stuck on idle, the Blend Space sample points may not be set up correctly.
- **Test montage playback:** Trigger a parry (calls Play Anim Montage). The parry animation should overlay on the locomotion. If nothing plays: confirm the Slot node in AnimGraph is named "DefaultSlot" (case-sensitive) and the montage's Slot Track also says "DefaultSlot".

EXPECTED RESULTS

- ✓ PASS — Speed variable = 0 when idle, ~600 when running
- ✓ PASS — Direction changes on strafing, correct anim blends in
- ✓ PASS — Parry montage plays over locomotion then blends back
- ✗ FAIL — Player has T-pose: ABP_ is not assigned on the Mesh component. Open BP_PlayerCharacter → select Mesh → Details → Animation → Anim Class → assign ABP_Player.
- ✗ FAIL — Montage doesn't play: Slot name mismatch. Open the AM_ montage asset and confirm the track name matches the Slot node name in the AnimGraph exactly.

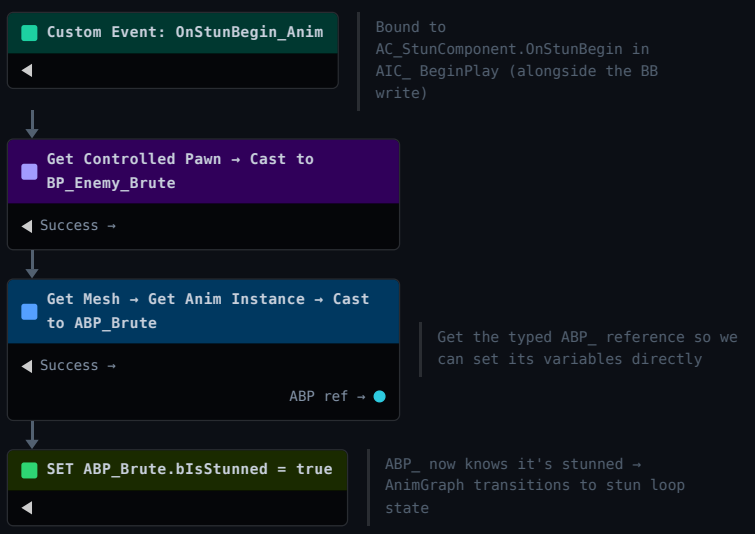
ABP_ENEMY + AIC_ STATE PUSH

Every enemy has its own ABP_. The pattern is identical to ABP_Player but enemies have additional state flags (bIsStunned, bIsDead, bArmourBroken) that the AI Controller writes in, not the ABP_ itself.

GRAPH ABP_[Enemy] Event Graph — variables + AIC_ push pattern

Create one ABP_ per enemy type (same process as ABP_Player). Each needs these variables: `Speed` (Float), `bIsStunned` (Bool), `bIsDead` (Bool), and any enemy-specific ones. The **AIC_ pushes state into these variables** rather than the ABP_ reading from components:

AIC_BRUTE — HOW IT PUSHES STATE TO ABP_ WHEN BLACKBOARD KEYS CHANGE



Do the same for `OnStunEnd` (SET bIsStunned=false), `OnArmourBroken` (SET bArmourBroken=true), and `OnDeath` (SET bIsDead=true). In the ABP_'s AnimGraph, use **State Machines** to transition between Locomotion, Stun, and Death states based on these booleans.

📌 **State Machine in AnimGraph:** Right-click the AnimGraph → Add New State Machine. Double-click to open it. Add States (right-click → Add State). Connect them with transition arrows. Click a transition arrow to set the condition — e.g. "bIsStunned = true" transitions from Locomotion to Stun state.

FLOW SUMMARY — ABP_ENEMY + AIC_ STATE PUSH

1. **ABP_ variables are the interface.** The AIC_ writes to them; the AnimGraph reads from them. Neither knows about the other's internals.
2. **AIC_ is the glue.** It binds to component dispatchers (OnStunBegin, OnDeath, OnArmourBroken) in BeginPlay and writes both the Blackboard AND the ABP_ variable in the same event handler.
3. **State Machine handles transitions.** Instead of a single Slot node, enemies use a State Machine with multiple states (Locomotion, Stun, Death) so the whole animation set changes on state transition.
4. **Death state is terminal.** The Death state has no outbound transitions — once bIsDead=true the enemy stays in the death animation until the ragdoll takes over.

🔧 DEBUG & TEST — ABP_ENEMY + AIC_ PUSH

HOW TO TEST

- **Watch ABP_ variables live:** Select an enemy during PIE → open its Anim Editor. The ABP_ Event Graph shows live variable values. Shoot the enemy — bIsStunned should flip to true during stun.
- **Test state machine transitions:** Open the ABP_'s AnimGraph → State Machine during PIE. The active state highlights green. Trigger a stun — the transition from Locomotion → Stun should visually activate.
- **Test death state terminal:** Kill the enemy. bIsDead=true → Death state activates. Try to stun the dead enemy — it should stay in Death state (no outbound transitions).

EXPECTED RESULTS

- ✓ PASS — bIsStunned flips in ABP_ the moment AC_StunComponent fires OnStunBegin
- ✓ PASS — State Machine: Locomotion → Stun → Locomotion on stun cycle
- ✓ PASS — Death state has no way back — enemy stays dead
- ✗ FAIL — ABP_ variable never changes: AIC_ BeginPlay bind isn't running. Confirm AIC_ BeginPlay fires (add a Print String) and that the bind call is wired correctly before the existing Blackboard write.

ANIM NOTIFIES — DAMAGE TIMING + BTT_ATTACKMELEE FIX

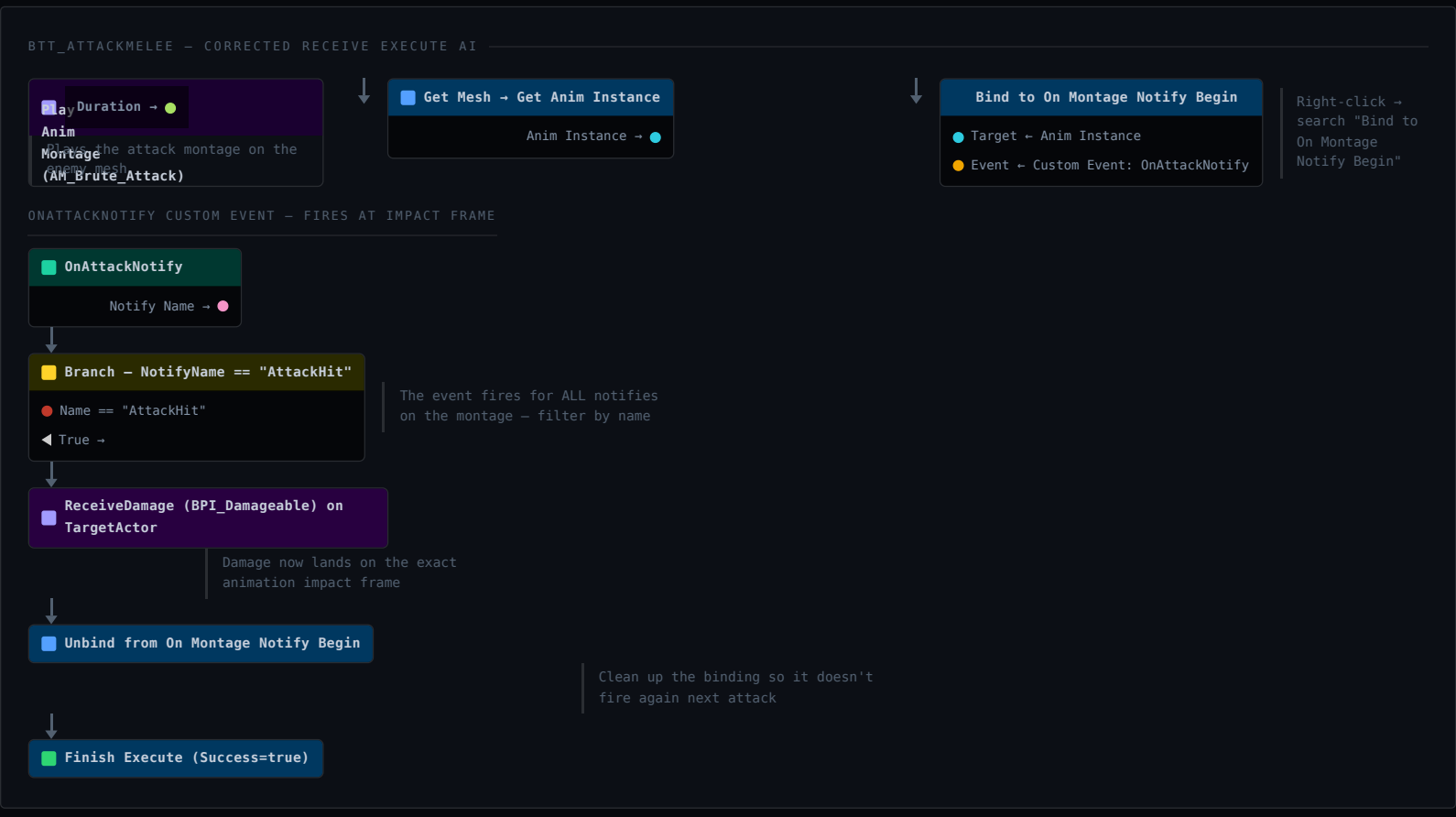
Part 1's BTT_AttackMelee applied damage at the end of the montage. The correct approach is to apply damage at the exact impact frame using an Anim Notify. This is a critical fix — without it, fast-moving enemies can deal damage during their backswing.

CREATE — Create AN_[Enemy]_AttackHit notify asset

In `Enemies/[EnemyType]/Animations/Notifies/` → right-click → **Animation → Anim Notify**. Name it `AN_Brute_AttackHit`. Open the `AM_Brute_Attack` montage asset. In the timeline at the bottom, right-click the Notifies track at the impact frame → **Add Notify → AN_Brute_AttackHit**. The notify appears as a small marker at that frame.

GRAPH — BTT_AttackMelee — corrected with notify-based damage

The Part 1 version applied damage on montage blend-out. Replace that with an `OnPlayMontageNotifyBegin` binding:



FLOW SUMMARY — ANIM NOTIFIES + BTT_ATTACKMELEE FIX

1. **Notify = event at a specific animation frame.** Placed directly on the timeline in the montage editor. The name you give it is the string that OnMontageNotifyBegin passes back.
2. **Filter by name.** OnMontageNotifyBegin fires for every notify on the montage. The Branch checking NotifyName == "AttackHit" ensures only the impact notify triggers damage, not footstep notifies or others on the same montage.
3. **Always unbind after use.** If you don't, the binding persists and fires on subsequent attacks, potentially dealing double damage.
4. **Finish Execute after damage.** The task is considered complete once damage is dealt — the BT can move on to the next node (e.g. an attack cooldown wait).

🔧 DEBUG & TEST — ANIM NOTIFIES

HOW TO TEST

- **Verify notify placement:** Open AM_Brute_Attack. Scrub the timeline to the impact frame. The AN_Brute_AttackHit marker should be visible on the Notifies track exactly at the moment the weapon visually connects.
- **Test damage timing:** Add a `Print String "DAMAGE AT: " + GetGameTimeSinceCreation` inside the True branch of the notify filter. Also print when the montage starts. The damage print should appear mid-animation, not at the end.
- **Test name filter:** Temporarily rename the notify to "WrongName" in the montage. Damage should stop being applied — confirming the filter works.
- **Test unbind cleanup:** Let the enemy attack 3 times. Each attack should deal damage exactly once. If damage multiplies each attack, the unbind is missing.

EXPECTED RESULTS

- ✓ PASS — Damage print appears mid-animation, not at end
- ✓ PASS — Wrong notify name: no damage applied
- ✓ PASS — 3 attacks → 3 damage applications, each exactly once
- ✗ FAIL — Damage multiplies each attack: Unbind is missing. Add "Unbind from On Montage Notify Begin" immediately after the damage call.
- ✗ FAIL — Notify never fires: check the notify name in the montage matches exactly (case-sensitive) the string in the Branch condition.

GRAPH AN_Player_FootstepL / R — notify implementation

Create `AN_Player_FootstepL` and `AN_Player_FootstepR` as Anim Notify assets. Open each one — it has a single function **Notify** to implement. Wire it:

NOTIFY FUNCTION — FIRES AT FOOT-STRIKE FRAME IN LOCOMOTION ANIMATION

📌 **ANS_ Anim Notify State — duration windows:** Unlike AN_ (a point in time), an **Anim Notify State** has a Begin and End event marking a *duration* on the animation. Used for the attack window: place `ANS_Brute_AttackWindow` spanning the full swing duration in AM_Brute_Attack. NotifyBegin sets `bAttackWindowOpen=true` on the enemy; NotifyEnd sets it false. Create one: right-click in Notifies folder → Animation → Anim Notify State. Implement `NotifyBegin` and `NotifyEnd` functions instead of the single `Notify` function. Place it by right-clicking the Notifies track in the montage → Add Notify State → pick your ANS_.

⚠️ **Physical Materials must be set up:** Assign Physical Material assets to the materials on your level geometry. In the Material asset → Physical Material override. Create Physical Material assets (right-click → Physics → Physical Material) and set their Surface Type to match your DT_FootstepSurfaces row names.

FLOW SUMMARY — FOOTSTEP SYSTEM

1. **Notify fires at exact foot-strike frame.** Open the locomotion animation and add the notify at the frame where the foot hits the ground — scrub to find it visually.
2. **Line trace reads the surface.** Traces 20 units down from the foot socket. Break HitResult gives the Physical Material, which gives the EPhysicalSurface enum value.
3. **DT_FootstepSurfaces maps surface to sound.** Convert the enum to a Name (the row key) and do a DataTable lookup to get the matching SC_ asset. Plays it at the impact location.
4. **Adding a new surface:** Create a Physical Material, assign to geometry, add one row to DT_FootstepSurfaces pointing to a new SC_ — zero notify changes needed.

🔧 DEBUG & TEST — FOOTSTEP SYSTEM

HOW TO TEST

- **Verify notify placement:** Open the walk/run animation. The footstep notifies should be visible on alternate frames corresponding to left and right foot strikes. Scrub the animation — the foot should be flat on the ground at that frame.
- **Test physical material detection:** Add `Print String "Surface: " + SurfaceName` after the Get Surface Type node. Walk on different surfaces — it should print different surface names.
- **Test sound plays:** Walk in-game. Footstep sounds should play at roughly the walking cadence. If silent: check the SC_ asset reference in DT_FootstepSurfaces isn't null.
- **Test default fallback:** Walk on a surface with no Physical Material assigned. The DT lookup will fail — add a fallback: if Row Not Found, play a default SC_Footstep_Default sound.

EXPECTED RESULTS

- ✓ PASS — Different surfaces print different surface type names
- ✓ PASS — Footstep sounds play at correct cadence with walking
- ✓ PASS — NS_Footstep_Dust spawns at each foot contact point
- ✗ FAIL — Always "Default" surface: Physical Materials not assigned to geometry. Select a floor mesh → Material slot → open the material asset → Physical Material property must point to your custom PhysicalMaterial asset.
- ✗ FAIL — No sound plays: the sound file inside the SC_ may be null. Open the SC_ asset and confirm an audio wave is connected to the Sound node.

HIT REACTIONS + DEATH RAGDOLL

GRAPH

Hit reaction — triggered from AC_EnemyHealthComponent

In BP_EnemyBase's BeginPlay, bind to `AC_EnemyHealthComponent.OnDamageTaken`. The hit direction determines which directional montage to play:

ONDAMAGETAKEN BINDING IN BP_ENEMYBASE

Custom Event: OnHit_React
Bound to AC_EnemyHealthComponent.OnDamageTaken in BeginPlay. Receives full FS_DamageInfo including HitLocation.
DamageInfo →

HitLocation − Actor Location → Normalize → Dot Product
A ← hit direction (normalised)
B ← Get Actor Forward Vector
DotResult (−1 to 1) →

Dot > 0.5 = hit from front. Dot < −0.5 = hit from behind. Cross product for left/right.

Branch − DotResult > 0.5 (front)
True →
False → check back/sides

Play Anim Montage (AM_Brute_HitReact_Front)
BlendInTime = 0.1

GRAPH

Death ragdoll — triggered from AC_EnemyHealthComponent.OnDeath

In BP_EnemyBase, bind to `AC_EnemyHealthComponent.OnDeath`:

ONDEATH_RAGDOLL CUSTOM EVENT IN BP_ENEMYBASE

Play Anim Montage (AM_[Enemy]_Death)

Brief death fall animation − 0.2s before ragdoll takes over

Set Timer → "ActivateRagdoll", Time=0.2s

ACTIVATERAGDOLL CUSTOM EVENT

Get Mesh → Set All Bodies Simulate Physics = true
New Simulate = true

Switches the skeletal mesh to full physics simulation

Get Mesh → Set Collision Profile Name = "Ragdoll"

Enable collision for the ragdoll bodies so it interacts with the world

Get Capsule Component → Set Collision Enabled = No Collision

Disable capsule so it doesn't block the player's movement through the corpse

Get Mesh → Add Impulse (Last Velocity × 1.5)
Impulse ← CharacterMovement.Velocity × 1.5
bVelChange = true

Carry momentum into the ragdoll so it flies in the direction the enemy was moving

Set Timer → "DissolveAndDestroy", Time=5.0s

Start 5-second countdown to dissolve and clean up the corpse

DISSOLVEANDDESTROY CUSTOM EVENT

FLOW SUMMARY — HIT REACTIONS + DEATH RAGDOLL

1. **Hit reaction direction:** Dot product of the hit direction vs actor forward gives a signed value. >0.5 = front, <−0.5 = back, cross product for left/right. Play the matching directional montage.
2. **Death is a 3-phase sequence:** Brief death montage (0.2s) → ragdoll activates → 5s timer → dissolve over 1.5s → Destroy Actor. Each phase hands off to the next via a timer.
3. **Ragdoll requires PHYS_ asset:** The PHYS_ asset defines which bones have physics bodies and their constraints. Without it, Set All Bodies Simulate Physics does nothing visible. Create it by opening the SK_ asset → Physics → Create Physics Asset.
4. **Capsule disabled on death:** Without this the invisible capsule blocks the player from walking through the corpse area after the enemy dies.

🔧 DEBUG & TEST — HIT REACTIONS + RAGDOLL

HOW TO TEST

- **Test hit reaction directions:** Shoot an enemy from the front, back, left, and right. A different montage should play each time. Add a `Print String "HIT DIR: " + DotResult` to see the raw dot product value.
- **Test ragdoll physics:** Kill an enemy moving at full speed. The corpse should carry momentum in the direction of movement when ragdoll activates. If it drops straight down, the Add Impulse call is missing or the velocity was already zero.
- **Test capsule disabled:** Kill an enemy. Try to walk through the corpse area. You should be able to without being blocked. If blocked, Set Collision Enabled isn't running.
- **Test dissolve cleanup:** Kill an enemy. Wait 5 seconds. It should start fading out, fully disappear after ~6.5s. Check the Outliner — the actor should be gone. If stuck visible, Destroy Actor isn't being called or the Timeline Finished event isn't wired.

EXPECTED RESULTS

- ✓ PASS — 4 different hit directions play 4 different montages
- ✓ PASS — Ragdoll carries momentum, falls naturally with physics
- ✓ PASS — Player can walk through the corpse area after death
- ✓ PASS — Corpse dissolves and disappears after ~6.5s
- ✗ FAIL — T-pose on death instead of ragdoll: PHYS_ asset not created or not assigned to the SK_ mesh. Open the SK_ asset → Physics → assign or create the PHYS_ there.
- ✗ FAIL — Enemy doesn't dissolve: Create Dynamic Material Instance must be called BEFORE setting parameters. Also ensure the master material has a "DissolveAmount" scalar parameter exposed.

IK RIG + FOOT IK IN ABP_PLAYER

Foot IK keeps feet planted on uneven terrain instead of floating above it. It's set up using the Two Bone IK node in the AnimGraph. The IK Rig asset defines the solver chains; the ABP_ uses them at runtime.

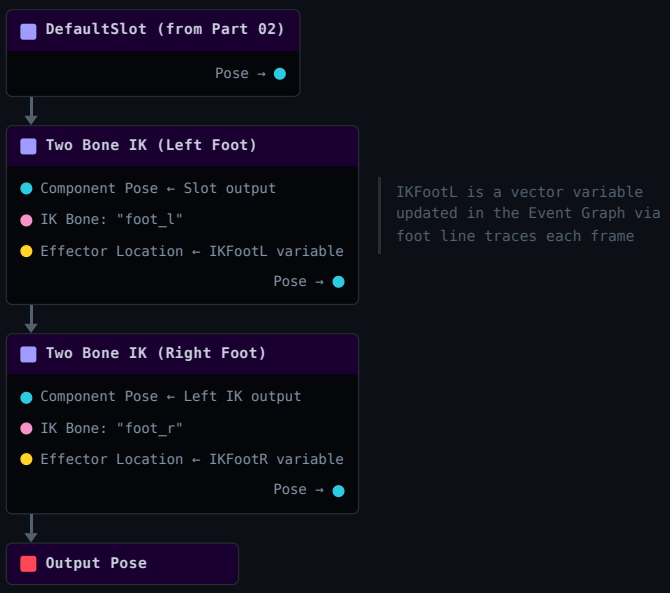
SETUP — Create IKR_Player and set up foot chains

In `Player/Mesh/` → right-click → **Animation → IK Rig**. Name it `IKR_Player`. Open it → assign `SK_Player` as the Preview Mesh. In the IK Retargeting panel, add two IK Chains: one for each leg starting at the thigh bone down to the foot bone.

ANIM GRAPH — Add Two Bone IK nodes in ABP_Player AnimGraph

In the ABP_Player AnimGraph, after the DefaultSlot node and before Output Pose, add Two Bone IK for each foot:

ANIMGRAPH — FOOT IK AFTER LOCOMOTION SLOT



IKFootL is a vector variable updated in the Event Graph via foot line traces each frame

In the ABP_ Event Graph, add foot traces to the Update Animation event — one per foot — line tracing from each foot socket down to the ground. Write the hit location into `IKFootL` and `IKFootR` vector variables. The Two Bone IK nodes read these to adjust foot placement each frame.

FLOW SUMMARY — IK RIG + FOOT IK

1. **Two Bone IK solves the leg chain.** Given a target foot position (from the trace), it bends the knee to reach it. No manual animation needed for terrain adaptation.
2. **Event Graph traces → AnimGraph solves.** The event graph does the traces and writes vector variables. The AnimGraph reads those variables into the IK node's Effector Location every frame.
3. **IK only activates when grounded.** Wrap the IK node in a Blend Boolean: bIsGrounded=true uses the IK pose, false uses the FK pose. No point solving foot IK while airborne.

🔧 DEBUG & TEST — FOOT IK

HOW TO TEST

- **Test on flat ground:** IK should produce no visible change on flat terrain — the feet already match the surface. If feet are floating or sunken, the trace distance or bone names are wrong.
- **Test on slopes:** Walk up a ramp. Feet should stay planted on the slope rather than floating above it. If both feet stay at the same height regardless of terrain, the traces aren't hitting the slope surface.
- **Draw debug traces:** In the Update Animation foot trace code, add Draw Debug Line (red, 0.02s duration) for each trace. You'll see the two foot trace lines each frame in the viewport.
- **Test airborne disable:** Jump. Feet should return to FK (animated) position mid-air. Land — IK re-activates. If IK stays on during jumps, the Blend Boolean using bIsGrounded isn't wired.

EXPECTED RESULTS

- ✓ PASS — Flat ground: no visible change, no floating
- ✓ PASS — Slopes: feet conform to surface angle
- ✓ PASS — Airborne: feet return to FK animated position
- ✗ FAIL — Feet twist in strange directions: the IK Bone names in the Two Bone IK node don't match the skeleton's actual bone names. Open SK_Player to confirm exact bone names.
- ✗ FAIL — No IK effect at all: the Effector Location Space may be wrong. Set it to "World Space" if using world-space trace hit locations.

POST PROCESS EFFECTS — PP_ENRAGE + PP_HITFLASH

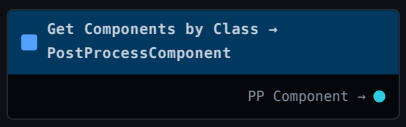
SETUP **Create Post Process Materials and assign to volume**

In `Materials/PostProcess/` → right-click → Material → name `PP_Enrage`. Open it → in the Details panel set **Material Domain = Post Process**. Add a **Scalar Parameter** node named `EnrageIntensity` (default 0). Build the effect: lerp between a normal scene colour and a red-tinted version based on EnrageIntensity. Do the same for `PP_HitFlash` using `HitFlashIntensity`.

Place a **Post Process Volume** in the level. Set Infinite Extent=true. In its Details → Rendering Features → Post Process Materials array → add both PP_ assets.

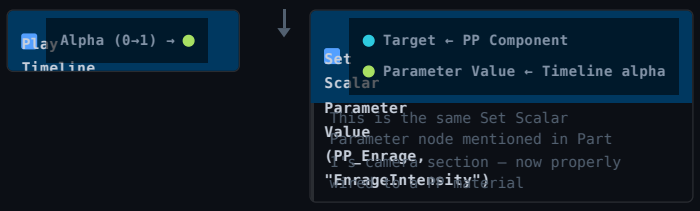
GRAPH **BP_PlayerCharacter — drive PP_Enrage from rage events**

ADD TO BEGINPLAY RAGE BINDINGS IN BP_PLAYERCHARACTER

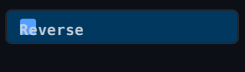


Or add a PostProcessComponent directly to BP_PlayerCharacter in the Components panel — gives local PP control to the player actor

ONRAGE_PPENRAGE CUSTOM EVENT (BOUND TO ONENRAGEDSTATEBEGIN)



ONRAGEEND_PPENRAGE (BOUND TO ONENRAGEDSTATEEND) — JUST REVERSE THE SAME TIMELINE



Reverses the same curve — fades the effect ba out. One Timeline asset, tw direction

FLOW SUMMARY — POST PROCESS EFFECTS

1. **Post Process Materials need Domain = Post Process.** This is set in the material's Details panel, not in the graph. Without i
2. **PP parameters driven by Timelines.** The Timeline generates a 0–1 float over time. Set Scalar Parameter Value on the PP Component pus Play/Reverse the same Timeline for enter/exit transitions.
3. **PP_HitFlash uses a spike curve.** Two keyframes: (0.0s, 0.0) and (0.075s, 1.0) and (0.15s, 0.0) with Cubic easing. The spike up a not a sustained effect.
4. **Adding a PostProcessComponent to the player** (rather than relying on the level volume) ensures the effects only apply to the loca would affect both.

🔧 DEBUG & TEST — POST PROCESS EFFECTS

HOW TO TEST

- **Test PP_Enrage ramp:** Print the Timeline alpha value each tick while it's playing. Trigger enrage — the value should go 0→1 over 0.2s. The screen should visibly shift colour. End enrage — should revert over the same time.
- **Test PP_HitFlash spike:** Take damage. The screen should flash white very briefly. Print the HitFlashIntensity parameter value — it should spike to 1.0 then return to 0 within 0.15s total.
- **Verify PP Volume settings:** Select the PostProcessVolume in the level. Confirm Infinite Extent (Unbound) is checked — without this the effect only applies inside the volume's bounds.
- **Test parameter name match:** Deliberately misspell the parameter name in Set Scalar Parameter Value. The effect should stop — confirming the name is case-sensitive and must match the material's parameter exactly.

EXPECTED RESULTS

- ✓ PASS — Screen shifts colour c
- ✓ PASS — Brief white flash on c
- ✓ PASS — Wrong parameter name:
- ✗ FAIL — No visible effect despit materials array. Open the are listed.
- ✗ FAIL — Effect is full intensity is 1 instead of 0. Open th Value = 0.

SOUND CLASSES + SOUND MIXES

> Sound Classes group audio into categories (SFX, UI, Music) so volume levels can be controlled independently. Sound Mixes m
> combat audio in cutscenes, etc.

SETUP

Create the Sound Class hierarchy

In `Audio/SoundClasses/` → right-click → **Sounds → Sound Class**. Create the hierarchy:

ASSET	PARENT	PURPOSE
SC_Master	None (root)	Global volume — changing this affects everything
SC_SFX	SC_Master	All gameplay sound effects
SC_UI	SC_Master	Menu and HUD audio
SC_Music	SC_Master	Background music tracks

Open
assi
cue

SETUP

Create Sound Mixes — SM_Gameplay + SM_Pause

In `Audio/SoundMixes/` → right-click → **Sounds → Sound Mix**. Create `SM_Pause`. Open it → in the Sound Class Effects array, add an entry: Sound Class = SC_SFX, Volume Adjuster = 0.1 (duck gameplay audio to 10% when paused). Apply the mix in BP_PlayerController when pausing:

WBP_PAUSEMENU — OPEN AND CLOSE WITH SOUND MIX

Push Sound Mix Modifier (SM_Pause)
- InSoundMixModifier ← SM_Pause

Call this when the pause menu opens — ducks gameplay SFX

↓

Set

ON RESUME

Pop Sound Mix Modifier (SM_Pause)
- InSoundMixModifier ← SM_Pause

Reverses the push — restores SFX volume to normal

↓

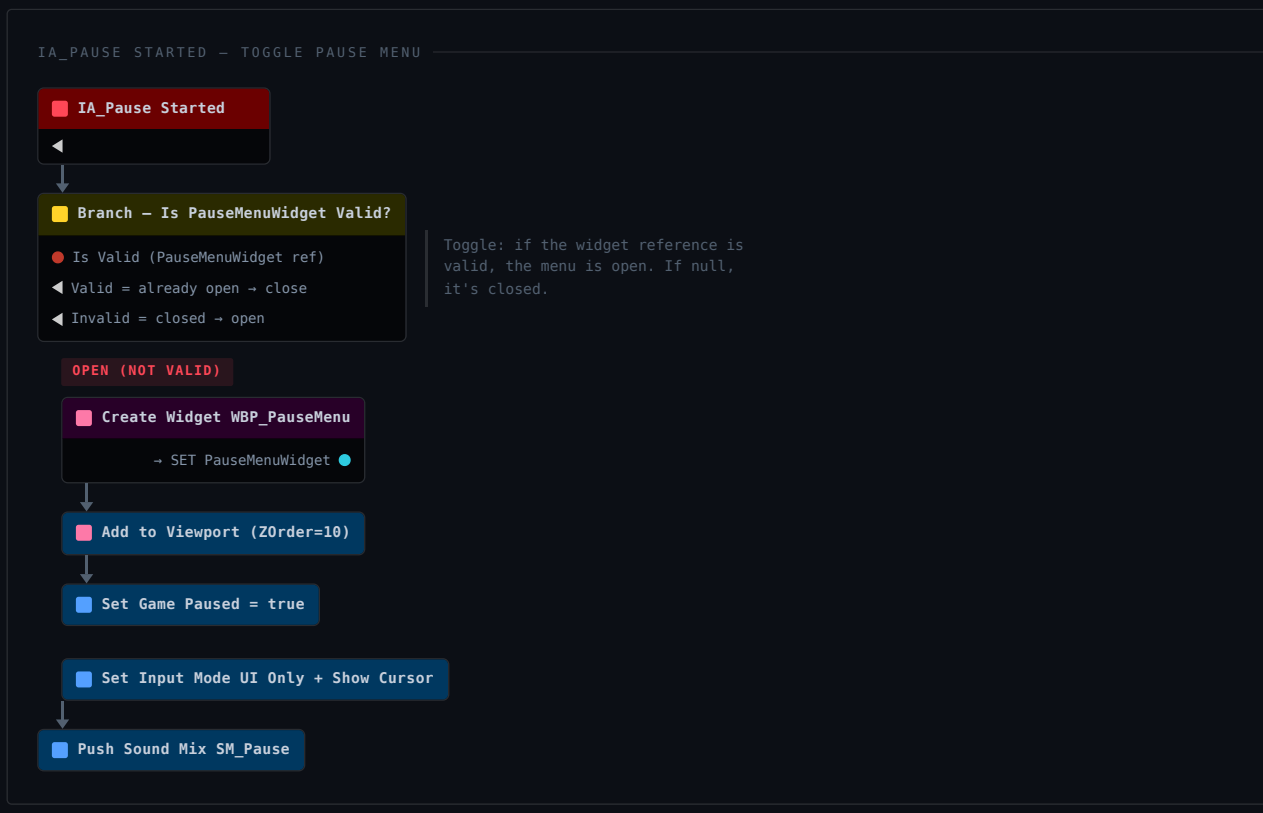
Set

FLOW SUMMARY — CLASSES + SOUND

1. **Sound Class = category.** Eve cue ass one Volu set Sour affe cue clas once
2. **Sound Mix = temporary modifier.** Push appl adju Pop then Adju are (mul so S Volu mean the curr volu
3. **Assign Sound Class to every Sound Cue.** Open ea cue ass Details Class. assignm sounds "Master default can't b individ contro

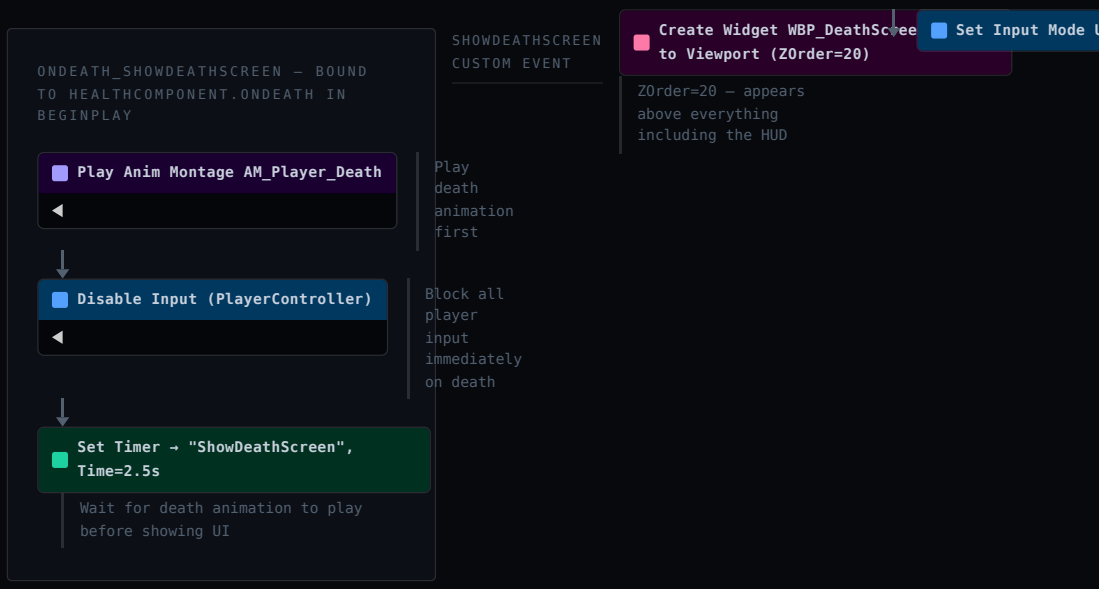
GRAPH **WBP_PauseMenu — full open/close wiring in BP_PlayerController**

First, add `IA_Pause` to IMC_Default bound to Escape/Start. In BP_PlayerController's Event Graph, add the IA_Pause event and a



GRAPH **WBP_DeathScreen — triggered by OnDeath after animation delay**

In BP_PlayerCharacter, add a third OnDeath binding alongside the existing ones:



Inside WBP_DeathScreen, the **Respawn** button calls `Open Level (by Name, current level)`. The **Main Menu** button calls `Open Level "L_MainMenu"`. Both use **Get Current Level Name** or a hardcoded string respectively.

GRAPH

WBP_LoadingScreen — owned by BP_GI_Anomaly

Add a function `ShowLoadingScreen` to BP_GI_Anomaly. Call it **before any Open Level call** anywhere in the project:

BP_GI_ANOMALY — SHOWLOADINGSCREEN FUNCTION

Create Widget WBP_LoadingScreen

● Owning Player ← Get First Player Controlle
→ SET LoadingScreenWidget ●

↓

Add to Viewport (ZOrder=100)

Highest ZOrder — covers absolutely everything during load

HIDELOADINGSCREEN — CALLED FROM BP_GM_ANOMALY BEGINPLAY IN THE NEW LEVEL

Get Game Instance → Cast to BP_GI_Anomaly

GI ref → ●

SET GI

↓

Remove from Parent (GI.LoadingScreenWidget)

New level has loaded and begun play — safe to remove the loading screen

FLOW SUMMARY — PAUSEMENU + DEATHSCREEN + LOADINGSCREEN

1. **PauseMenu is a toggle.** Storing the widget reference in a variable lets you check "is the menu open?" without a separate bool — if the ref is valid, it's open. SET to null on close.
2. **DeathScreen waits for the animation.** 2.5s timer gives the death montage time to play before the UI fades in. Disable Input immediately on death so the player can't keep firing during the animation.
3. **LoadingScreen lives on the Game Instance** because it must survive the level transition — the old level's actors (including PlayerController) are destroyed during Open Level. Only the Game Instance persists to remove the widget.
4. **ZOrder hierarchy:** HUD=0, SkillMenu=1, PauseMenu=10, DeathScreen=20, LoadingScreen=100. Each layer of urgency sits above the last.

🔧 DEBUG & TEST — PAUSE, DEATH, LOADING SCREENS

HOW TO TEST

- **Test pause toggle:** Press Escape → menu opens (game frozen, cursor visible, SFX ducked). Press Escape again → menu closes (game resumes, cursor hidden, SFX normal). Check the PauseMenuWidget variable is null after closing.
- **Test death flow:** Die in-game. Death animation should play for 2.5 seconds. DeathScreen should fade in. All inputs should be blocked during the animation. Both buttons (Respawn and Main Menu) should function.
- **Test loading screen persistence:** Call ShowLoadingScreen then immediately Open Level. The loading screen should stay visible during the transition. In the new level's BP_GM_Anomaly BeginPlay, print "NEW LEVEL READY" — the loading screen should disappear shortly after.
- **Test ZOrder stacking:** Open the pause menu with the HUD visible. Pause menu should cover the HUD. Open skill menu (if accessible) — pause menu should still be on top.

EXPECTED RESULTS

- ✓ PASS — Pause toggles correctly, SFX ducks/restores
- ✓ PASS — Death: 2.5s animation → DeathScreen fades in → buttons work
- ✓ PASS — Loading screen visible during entire level transition
- ✓ PASS — Correct ZOrder: HUD beneath Pause beneath Death beneath Loading
- ✗ FAIL — Game doesn't freeze on pause: Set Game Paused = true is missing or not executing. Add a print immediately before it to confirm the code path runs.
- ✗ FAIL — Loading screen disappears instantly: HideLoadingScreen is being called before the new level is ready. Move it to BP_GM_Anomaly BeginPlay in the destination level, not in the source level.